Augmented ESKF With Residual Mount Misalignment State

Codex

April 11, 2026

1 Purpose

This note documents the current augmented error-state Kalman filter (ESKF) formulation used in the project after adding a residual mount-misalignment state. The important implementation idea is:

- the align filter supplies a coarse IMU-to-vehicle mount estimate,
- that coarse mount is frozen when ESKF initializes,
- raw IMU samples are pre-rotated by the frozen coarse mount before ESKF propagation,
- ESKF carries an internal residual fine mount quaternion q_{cs},
- body/vehicle-frame velocity measurements use q_{cs},
- GNSS and stationary-gravity measurements do not directly observe q_{cs}.

The document also records the coordinate conventions and the mount-state update equations, since the current observation that loose and augmented ESKF drift toward similar wrong mount values points to a possible shared modeling-convention error.

2 Coordinate Conventions

2.1 Frames

Symbol	Meaning
n	Navigation frame. The implementation uses local NED: x north, y east, z down.
b	Raw IMU/body sensor frame. This is the frame of ESF-RAW/ESF-MEAS samples before mount correction.
v	Vehicle frame. Internally this is treated as forward-right-down (FRD): x forward, y right/lateral, z down.
s	Seeded vehicle frame. Raw IMU samples are rotated from b into s using the frozen coarse align mount. If the coarse mount is perfect, $s = v$.
c	Corrected vehicle frame after applying ESKF residual mount. If the residual mount is perfect, $c = v$.

2.2 Rotation and Quaternion Convention

The document uses active rotation matrices C_{ab} that map coordinates of a vector from frame b to frame a:

$$x_a = C_{ab} x_b.$$

The quaternion q_{ab} denotes the same rotation as C_{ab}. Quaternion multiplication composes rotations left-to-right in the usual matrix sense:

$$C(q_1 \otimes q_2) = C(q_1)C(q_2).$$

For a small angle $\delta\theta \in \mathbb{R}^3$,

$$\delta q(\delta\theta) \approx \begin{bmatrix} 1 & \frac{1}{2}\delta\theta^T \end{bmatrix}^T, \qquad C(\delta q) \approx I + [\delta\theta]_\times .$$

2.3 Coarse Mount Seed

The align filter state stores a mount quaternion named q_{vb} in code. It maps vehicle-frame vectors into raw body-frame vectors:

$$x_b = C_{bv}(q_{vb}) x_v.$$

At ESKF initialization, the current align mount is frozen:

$$q_{vb}^0 \leftarrow \hat{q}_{vb,\text{align}}(t_{\text{init}}).$$

After that point, ESKF prediction uses this frozen seed even if align continues changing. Raw IMU increments are pre-rotated into the seeded frame:

$$\Delta\theta_s = C_{sb}(q_{vb}^0)\Delta\theta_b, \qquad \Delta v_s = C_{sb}(q_{vb}^0)\Delta v_b,$$

where, under the code convention,

$$C_{sb}(q_{vb}^0) = C_{bv}(q_{vb}^0)^T.$$

3 Symbol Table

Symbol	Definition
q_{ns}	ESKF nominal attitude quaternion. Its rotation C_{ns} maps seeded-frame vectors into NED. In code this is the nominal quaternion fields $q0, q1, q2, q3$, exposed as q_bn.
v_n	Velocity in NED, $[v_N, v_E, v_D]^T$.
p_n	Position in the local NED tangent frame, $[p_N, p_E, p_D]^T$.
b_g	Gyroscope bias in seeded frame, radians/s.
b_a	Accelerometer bias in seeded frame, m/s^2.
q_{cs}	Residual fine mount quaternion. Its rotation C_{cs} maps seeded-frame vectors into corrected vehicle frame. Initialized to identity.
$\delta\theta_s$	Small attitude error state. It is right-injected into q_{ns}.
δv_n	Velocity error state.

δp_n	Position error state.
δb_g	Gyro-bias error state.
δb_a	Accel-bias error state.
$\delta\psi_{cs}$	Residual mount small-angle error state. It is left-injected into q_{cs}.
g_n	Gravity vector in NED, approximately $[0, 0, g]^T$, with down positive.
Q_g, Q_a	Gyro and accel white-noise variances used in covariance prediction.
Q_{bg}, Q_{ba}	Gyro-bias and accel-bias random-walk variances.
Q_m	Residual mount random-walk variance, code field `mount_align_rw_var`.

4 Nominal and Error States

4.1 Nominal State

The augmented nominal state has 20 scalar components:

$$x = \begin{bmatrix} q_{ns} \\ v_n \\ p_n \\ b_g \\ b_a \\ q_{cs} \end{bmatrix}, \qquad q_{ns}, q_{cs} \in \mathbb{S}^3.$$

Expanded by dimensions:

$$x \in \mathbb{S}^3 \times \mathbb{R}^3 \times \mathbb{R}^3 \times \mathbb{R}^3 \times \mathbb{R}^3 \times \mathbb{S}^3.$$

4.2 Error State

The ESKF error state has 18 scalar components:

$$\delta x = \begin{bmatrix} \delta\theta_s \\ \delta v_n \\ \delta p_n \\ \delta b_g \\ \delta b_a \\ \delta\psi_{cs} \end{bmatrix} \in \mathbb{R}^{18}.$$

The ordering in C/Rust is:

Index range	State
0..2	$\delta\theta_s$
3..5	δv_n
6..8	δp_n
9..11	δb_g
12..14	δb_a
15..17	$\delta\psi_{cs}$

5 Nominal Propagation

The IMU delta supplied to ESKF has already been rotated into the frozen seeded frame s:

$$\Delta\theta_s, \quad \Delta v_s, \quad \Delta t.$$

Bias-corrected increments are:

$$\Delta\theta_s^c = \Delta\theta_s - b_g\Delta t, \qquad \Delta v_s^c = \Delta v_s - b_a\Delta t.$$

The nominal attitude update is:

$$q_{ns}^+ = q_{ns} \otimes \delta q(\Delta\theta_s^c), \qquad q_{ns}^+ \leftarrow \text{normalize}(q_{ns}^+).$$

The nominal velocity update is:

$$v_n^+ = v_n + C_{ns}(q_{ns}^+)\Delta v_s^c + g_n\Delta t.$$

The nominal position update is implemented with the updated velocity:

$$p_n^+ = p_n + v_n^+\Delta t.$$

Biases are constant in the nominal propagation:

$$b_g^+ = b_g, \qquad b_a^+ = b_a.$$

The residual mount quaternion is also constant in the nominal propagation:

$$q_{cs}^+ = q_{cs}.$$

6 Error-State Propagation

The linearized error propagation has the standard ESKF form:

$$\delta x_{k+1} = F_k\delta x_k + G_k w_k.$$

The process-noise vector has 15 components:

$$w = \begin{bmatrix} n_g \\ n_a \\ n_{bg} \\ n_{ba} \\ n_m \end{bmatrix} \in \mathbb{R}^{15}.$$

The covariance prediction is:

$$P_{k+1}^- = F_k P_k^+ F_k^T + G_k Q_k G_k^T.$$

The diagonal noise covariance used by the C implementation is:

$$Q_k = \text{diag}\left(\begin{array}{ccc} \sigma_g^2\Delta t^2 I_3, & \sigma_a^2\Delta t^2 I_3, & \sigma_{bg}^2\Delta t I_3, \\ \sigma_{ba}^2\Delta t I_3, & \sigma_m^2\Delta t I_3 & \end{array} \right).$$

The residual mount error is modeled as a random walk:

$$\delta\psi_{cs,k+1} = \delta\psi_{cs,k} + n_{m,k}.$$

There is no modeled coupling from $\delta\psi_{cs}$ into mechanization because the IMU has already been pre-rotated by the frozen coarse seed. Therefore, in the current architecture, residual mount is intended to be observable through measurement models, not through propagation.

7 Measurement Update

For each scalar measurement, the implementation uses a standard sequential Kalman update:

$$r = z - h(\hat{x}),$$

$$S = HPH^T + R, \qquad K = PH^T S^{-1},$$

$$\delta x = Kr, \qquad P^+ = P^- - SKK^T.$$

After computing δx, the error is injected into the nominal state and the covariance is reset as described in Section 8.

7.1 GNSS Position

The position measurement model is:

$$z_p = p_n + \eta_p.$$

For scalar components:

$$h_{p_i}(x) = e_i^T p_n, \qquad i \in \{N, E, D\}.$$

The residual mount Jacobian block is zero:

$$\frac{\partial h_p}{\partial \delta\psi_{cs}} = 0.$$

7.2 GNSS Velocity

The velocity measurement model is:

$$z_v = v_n + \eta_v.$$

For scalar components:

$$h_{v_i}(x) = e_i^T v_n, \qquad i \in \{N, E, D\}.$$

The residual mount Jacobian block is also zero:

$$\frac{\partial h_v}{\partial \delta\psi_{cs}} = 0.$$

7.3 Vehicle Velocity and Non-Holonomic Constraint

The seeded-frame velocity predicted from the navigation state is:

$$v_s = C_{ns}(q_{ns})^T v_n.$$

The corrected vehicle-frame velocity is:

$$v_c = C_{cs}(q_{cs}) v_s.$$

Because the runtime pre-rotates raw IMU samples by the frozen coarse seed q_{vb}^0 before ESKF prediction, q_{cs} is a seeded-frame-to-vehicle correction. The physical vehicle-to-body mount reported for comparison is therefore

$$q_{vb} = q_{vb}^0 \otimes q_{cs}^{-1},$$

not $q_{cs} \otimes q_{vb}^0$. The latter composition is only a legacy diagnostic and will appear with the wrong sign when the seed yaw error is absorbed by q_{cs}.

The body/vehicle speed measurement along x is:

$$h_{\text{speed},x}(x) = e_x^T v_c.$$

The non-holonomic lateral and vertical constraints are:

$$h_{\text{nhc},y}(x) = e_y^T v_c \approx 0, \qquad h_{\text{nhc},z}(x) = e_z^T v_c \approx 0.$$

For a left perturbation of the residual mount,

$$q_{cs}^{\text{true}} = \delta q(\delta\psi_{cs}) \otimes q_{cs},$$

the perturbed velocity is:

$$v_c^{\text{true}} = C(\delta q(\delta\psi_{cs}))C_{cs}v_s \approx (I + [\delta\psi_{cs}]_\times)v_c = v_c - [v_c]_\times \, \delta\psi_{cs}.$$

Therefore the residual mount block for any scalar vehicle-velocity component $e_i^T v_c$ is:

$$\frac{\partial(e_i^T v_c)}{\partial\delta\psi_{cs}} = -e_i^T \, [v_c]_\times \, .$$

This is the intended structural role of the added mount states: lateral and vertical velocity residuals can move $\delta\psi_{cs}$ rather than forcing all residual mount error into attitude, velocity, and biases.

7.4 Stationary Gravity

During stationary or low-dynamic periods, the current ESKF stationary-gravity update uses accelerometer measurements already expressed in the seeded frame. The simplified model is:

$$a_s \approx -C_{ns}(q_{ns})^T g_n + b_a + \eta_a.$$

The implementation currently updates only the x and y accelerometer channels:

$$h_{\text{grav},x}(x) = e_x^T \left(-C_{ns}^T g_n + b_a\right),$$

$$h_{\text{grav},y}(x) = e_y^T \left(-C_{ns}^T g_n + b_a\right).$$

In the generated stationary-gravity rows, the residual mount block is zero:

$$\frac{\partial h_{\text{grav}}}{\partial\delta\psi_{cs}} = 0.$$

This is deliberate under the current architecture because the stationary accelerometer vector has already been transformed into the seeded frame by the frozen coarse mount. If residual mount is also applied in this measurement without revisiting the frame definition, the correction can be double-counted or applied in the wrong frame.

7.5 Zero Velocity

The current zero-velocity update fuses zero velocity in the navigation frame:

$$z_v = 0, \qquad h_v(x) = v_n.$$

Therefore zero velocity does not directly observe residual mount:

$$\frac{\partial h_{\text{zupt}}}{\partial\delta\psi_{cs}} = 0.$$

This differs from a body-frame zero-velocity constraint, which would use v_c and would directly update q_{cs}.

8 Error Injection and Covariance Reset

8.1 Attitude Injection

The main attitude error is right-injected:

$$q_{ns}^{+} = q_{ns}^{-} \otimes \delta q(\delta\theta_s),$$

followed by normalization.

8.2 Translational and Bias Injection

The vector states are additively corrected:

$$\begin{aligned} v_n^{+} &= v_n^{-} + \delta v_n, & p_n^{+} &= p_n^{-} + \delta p_n, \\ b_g^{+} &= b_g^{-} + \delta b_g, & b_a^{+} &= b_a^{-} + \delta b_a. \end{aligned}$$

8.3 Residual Mount Injection

The residual mount error is left-injected:

$$q_{cs}^{+} = \delta q(\delta\psi_{cs}) \otimes q_{cs}^{-},$$

followed by normalization.

The injection side is the same as loose currently uses for its residual mount state, but the ESKF residual has a different reporting composition because the runtime has already applied the frozen coarse mount to the IMU before ESKF propagation.

8.4 Covariance Reset

After injection, the error definition has changed, so the covariance is reset. The attitude reset uses the small-angle SO(3) reset Jacobian:

$$G_\theta \approx I - \frac{1}{2}\left[\delta\theta_s\right]_\times .$$

The same block form is applied to the residual mount error:

$$G_\psi \approx I - \frac{1}{2}\left[\delta\psi_{cs}\right]_\times .$$

The full reset matrix is block diagonal except for identity blocks on non-rotational states:

$$G_{\text{reset}} = \text{blkdiag}\left(G_\theta, I_3, I_3, I_3, I_3, G_\psi\right).$$

Then:

$$P^{+} \leftarrow G_{\text{reset}} P^{+} G_{\text{reset}}^{T}.$$

9 Initialization

9.1 Coarse Mount Freeze

When ESKF initializes from GNSS, the current coarse mount estimate is copied into a frozen ESKF mount seed:

$$q_{vb}^{0} \leftarrow \hat{q}_{vb,\text{align}}.$$

Subsequent ESKF IMU prediction uses q_{vb}^{0}, not the live align output.

9.2 Residual Mount Initial Condition

The residual mount nominal starts at identity:

$$q_{cs,0} = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}^T.$$

The residual mount covariance is initialized to a 5° one-sigma uncertainty on each axis:

$$P_{\psi\psi,0} = \left(5\frac{\pi}{180}\right)^2 I_3.$$

10 Full Mount for Plotting and Comparison

The current visualizer compares ESKF residual mount estimates to ESF-ALG by composing the frozen seed and the residual estimate similarly to the current loose plot:

$$q_{\text{full},vb} = q_{cs} \otimes q_{vb}^0.$$

That full mount quaternion is then converted from the internal FRD convention to the ESF-ALG FLU convention before extracting roll, pitch, and yaw:

$$q_{\text{full,FLU}} = \text{FRD_to_FLU}\left(q_{\text{full},vb}\right).$$

The plotted ESKF mount angles are:

$$(\phi, \theta, \psi)_{\text{ESKF plot}} = \text{RPY}_{\text{ALG}}\left(q_{\text{full,FLU}}\right).$$

This plot convention is useful diagnostically, but it is also a high-priority audit surface. The Open Aided Navigation loose formulation defines C_{cs} as a sensor-to-car rotation. Under the ESKF seed architecture,

$$C_{sb} = (C_{bv}^0)^T, \qquad C_{cb} = C_{cs}C_{sb} = C_{cs}(C_{bv}^0)^T.$$

Therefore, if the quantity to compare against ESF-ALG is a car/vehicle-to-body mount quaternion q_{vb}, the physically corresponding rotation matrix is

$$C_{bc} = C_{cb}^T = C_{bv}^0 C_{cs}^T.$$

Under the quaternion convention used in this document, that corresponds to

$$q_{\text{full},vb} = q_{vb}^0 \otimes q_{sc}, \qquad q_{sc} = q_{cs}^{-1}.$$

Thus the existing plot composition $q_{cs} \otimes q_{vb}^0$ should not be treated as reference-correct unless q_{cs} is intentionally defined as the inverse residual and/or the seed quaternion has the opposite direction. This is a likely source of misleading ESKF-vs-ESF-ALG and loose-vs-ESF-ALG mount plots.

11 Current Modeling Assumptions to Audit

Both loose and augmented ESKF have been observed to estimate mount angles in a similar wrong direction and magnitude. Since the two filters use different state sizes and propagation details, this points to a shared convention or measurement-model issue. The highest-priority checks are:

1. **Residual mount direction.** Verify whether q_{cs} should map $s \to c$, $c \to s$, or the inverse under the code's quaternion convention.

2. **Composition order.** The plotted full mount currently uses $q_{cs} \otimes q_{vb}^0$. If q_{cs} follows the Open Aided Navigation C_{cs} convention, the physically correct q_{vb} comparison should instead use $q_{vb}^0 \otimes q_{cs}^{-1}$.

3. **NHC residual frame.** The current vehicle velocity prediction is $v_c = C_{cs} C_{ns}^T v_n$. If C_{cs} is the inverse of the intended correction, the NHC Jacobian will push mount in the wrong direction while still reducing the scalar residual.

4. **Left-versus-right perturbation.** The mount Jacobian above assumes left injection:

$$q_{cs}^{\text{true}} = \delta q(\delta\psi_{cs}) \otimes q_{cs}.$$

   If the generated symbolic model or the physical interpretation assumes right perturbation, the Jacobian sign and frame are wrong.

5. **Pre-rotation plus residual correction.** Because IMU samples are already pre-rotated by the frozen coarse mount, residual mount should not be applied again in propagation. It should enter only measurements whose predicted quantity is expressed in corrected vehicle coordinates.

6. **Vehicle-axis signs.** Confirm that the NHC signs match the internal FRD frame: forward $+x$, right $+y$, down $+z$. A FLU/FRD mismatch can make roll/yaw estimates look plausible in magnitude while converging to the wrong convention.

12 Body-Velocity/NHC Mount-Yaw Investigation

This section records the diagnostic result from the April 2026 replay investigation on `logger/data/ubx_raw_20260406_155424.bin`. The purpose was to determine whether the wrong residual mount-yaw convergence is caused by GNSS updates, by the lateral NHC row, by the vertical NHC row, or by a shared convention/coupling issue.

12.1 Diagnostic Setup

After the earlier GNSS-to-mount experiment, GNSS position and velocity updates were prevented from injecting residual mount states by zeroing $dx_{15:17}$ for GNSS rows before nominal injection. The covariance update was left unchanged. With that diagnostic in place, the final ESKF plotted mount yaw still settled near the same wrong value:

$$\psi_{\text{full,ESKF}} \approx 187.55^\circ.$$

Two additional compile-time diagnostic gates were then added:

- `SF_ESKF_DIAG_DISABLE_BODY_VEL_Y_MOUNT`
- `SF_ESKF_DIAG_DISABLE_BODY_VEL_Z_MOUNT`

Each gate leaves the corresponding scalar body-velocity measurement active for non-mount states and covariance, but zeros $dx_{15:17}$ before residual mount injection. This makes the test a nominal-state mount-injection isolation test, not a full measurement removal test.

The visualizer was also extended to plot cumulative innovation sums and absolute innovation sums by update cue. These traces should be read together with the existing cumulative residual mount-yaw increment traces.

12.2 Replay Results

The baseline command was:

```
cargo run -p sim -bin visualizer - logger/data/ubx_raw_20260406_155424.bin
                                   -profile-only -misalignment align
                                   -dump-loose-time-s 1200 -dump-window-s 0.05.
```

At $t \approx 1200$ s, with GNSS mount injection blocked but body-velocity mount injection enabled:

Cue	Yaw dx sum	Yaw dx abs	Innovation sum	Innovation abs
`body_vel_y`	$+3.213054°$	$29.093795°$	-859.557739	5600.595703
`body_vel_z`	$+3.272938°$	$8.319831°$	-38.995468	5209.825684

The one-row-disabled variants showed strong substitution between the two NHC rows:

Variant	Final ESKF yaw	`body_vel_y` yaw dx	`body_vel_z` yaw dx
baseline	$187.553662°$	$+3.213054°$	$+3.272938°$
disable `body_vel_y` mount injection	$186.571367°$	$0.000000°$	$+6.995455°$
disable `body_vel_z` mount injection	$187.710707°$	$+6.313429°$	$0.000000°$
disable both `body_vel_y/z` mount injection	$176.714426°$	$0.000000°$	$0.000000°$

When both lateral and vertical NHC mount injections were disabled, the remaining residual mount-yaw motion was dominated by zero-velocity north/east updates, about $-4.444071°$, plus smaller stationary and vertical terms.

12.3 Interpretation

The wrong positive residual mount-yaw shift requires body-velocity/NHC mount injection. It is not isolated to one scalar row:

- disabling only `body_vel_y` transfers most of the yaw correction into `body_vel_z`;
- disabling only `body_vel_z` transfers most of the yaw correction into `body_vel_y`;
- disabling both rows removes the positive wrong-yaw mode.

This points to a shared NHC/covariance/convention issue rather than a single bad scalar residual. The simple identity, forward-motion lateral-row check gives

$$h_y \approx v_y + \psi_{cs,z} v_x, \qquad z - h_y = -h_y, \qquad H_{\psi_{cs,z}} \approx v_x.$$

For $v_x > 0$, a negative lateral innovation would normally produce a negative direct yaw correction if the direct mount-yaw gain dominates. In the replay, however, both NHC innovation sums are negative while the cumulative residual mount-yaw increments are positive. Therefore the effective replay gain from NHC innovation to residual mount yaw is negative. The practical conclusion is that the real log is being driven by coupled covariance structure and/or residual-mount convention, not by the simple direct lateral Jacobian term alone.

12.4 Convention Check Result

The convention A/B test showed that $q_{vb}^0 \otimes q_{cs}^{-1}$ is the correct physical ESKF mount composition for the current runtime. In noisy gnss-ins-sim replay, the old plotted composition made the ESKF yaw appear to move in the wrong direction, while the corrected composition tracks the true mount much better:

$$q_{\text{full}} = q_{vb}^0 \otimes q_{cs}^{-1}.$$

This means the lateral NHC row was not necessarily pushing the physical mount the wrong way in the simulation runs; the main error was the diagnostic/reporting composition. The inverse-residual NHC model remains a useful diagnostic only if future real-log replays still show disagreement after plotting and evaluation use the corrected composition.

12.5 Remaining Experiments

1. **Keep both full-mount compositions visible during debug.** Use the physical ESKF composition,

   $$q_{\text{full}} = q_{vb}^0 \otimes q_{cs}^{-1},$$

   as the primary trace, and retain the legacy composition,

   $$q_{\text{legacy}} = q_{cs} \otimes q_{vb}^0,$$

   only as a sign/convention diagnostic.

2. **Test inverse residual mount in the NHC measurement model.** Compare the current prediction

   $$v_c = C_{cs} C_{ns}^T v_n$$

   against the inverse-residual variant

   $$v_c = C_{cs}^T C_{ns}^T v_n.$$

   This should be implemented by changing the symbolic model and regenerating the Jacobians, not by only flipping an injected dx_{17}. Flipping dx_{17} alone is useful as a quick sign diagnostic, but it makes the covariance update inconsistent with the measurement model.

3. **Verify perturbation side and reset convention.** The generated NHC Jacobian assumes left injection,

   $$q_{cs}^{\text{true}} = \delta q(\delta\psi_{cs}) \otimes q_{cs}.$$

   If the inverse-residual model is selected, the perturbation definition and covariance reset block must be audited with the same convention.

4. **Replay the same log with the A/B models.** For each variant, compare final mount yaw, NHC innovation sums, NHC absolute innovation sums, and cumulative $\delta\psi_{cs,z}$ by cue. The preferred model should reduce the NHC residuals while moving the plotted physical mount toward ESF-ALG in the same convention.

13 Implementation Cross-Reference

File / function	Role
`ekf/eskf.py`	Symbolic ESKF generation. Defines the 18-state error model and generated observation Jacobians.
`sf_eskf_predict()`	Propagates nominal state and covariance, including mount random-walk noise.
`sf_eskf_nominal_vehicle_velocity()`	Computes $v_c = C_{cs} C_{ns}^T v_n$.
`sf_eskf_fuse_body_vel()`	Applies lateral/vertical NHC updates using v_c.
`sf_eskf_fuse_body_speed_x()`	Applies forward wheel/body-speed update using v_c.
`sf_eskf_fuse_stationary_gravity()`	Applies stationary gravity update without residual mount columns.
`sf_eskf_inject_error_state()`	Injects $\delta\theta_s$ into q_{ns} and $\delta\psi_{cs}$ into q_{cs}.
`sf_eskf_apply_reset()`	Applies SO(3) reset blocks to attitude and residual mount covariance blocks.
`sensor_fusion.c`	Freezes coarse align mount at ESKF initialization and pre-rotates IMU samples by that frozen seed.
`visualizer/pipeline/ekf_compare.rs`	Plots ESF-ALG versus frozen seed plus ESKF residual mount.